UNIX, THE SHELL, AND MAKEFILES

Course notes: the Unix shell, filesystems, permissions, processes, C compilation, and make

These notes use a Unix/Linux terminal running a shell such as Bash. A **command** is a program or shell operation invoked by entering text at the prompt. The shell interprets the command and reports output or errors. The examples are designed for an ordinary user account; administrator privileges and external data files are not required.

In command examples, `$` denotes the prompt and is **not** typed. A line without `$` may represent text entered into a running program or sample output. Replace `student` in illustrative paths with the account name on the system.

Structure of a command

A command normally consists of a command name, optional flags, and any required arguments. Spaces separate these components. For example, in `ls -a ~/unix_lab`, `ls` is the command, `-a` is an option, and `~/unix_lab` is a path argument. Commands are entered after the prompt and executed by pressing **Enter**. The shell is case-sensitive.

UNIT 1 | Directories and pathnames

The working directory

The filesystem is organized as a hierarchy of directories. A **directory** holds named entries for files and subdirectories. Every shell has a **current working directory**, which is the starting location used to interpret relative filenames. Each user also has a **home directory**.

```
$ pwd
/home/student
$ ls
$ ls -a
```

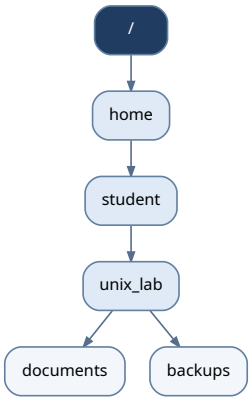
`pwd` prints the absolute path of the working directory. `ls` lists the visible names in a directory; `ls -a` includes names beginning with a dot, such as configuration files. A blank result from `ls` may mean that no visible entries exist.

The filesystem root is named `/`. It is distinct from the home directory. On many Linux systems a home path looks like `/home/student`, but the actual path depends on the system.

Creating and navigating directories

```
$ cd
$ mkdir unix_lab
$ cd unix_lab
$ mkdir documents
$ mkdir backups
$ pwd
/home/student/unix_lab
$ ls
backups  documents
```

`mkdir` creates a directory. `cd directory` changes the working directory; `cd` with no argument returns to the home directory. The displayed output is illustrative and assumes the account is named `student`.



Directory hierarchy and parent-child relationships.

Two special path components are available in every directory: `.` refers to the current directory; `..` refers to its parent. A slash separates directory components in a path.

```
$ cd documents
$ pwd
/home/student/unix_lab/documents
$ cd ..
$ cd ./backups
$ cd ../documents
$ cd ~
$ cd ~/unix_lab
```

The `~` notation is expanded by the shell to the user's home directory. Thus `~/unix_lab` refers to the same directory regardless of the working directory.

Absolute and relative paths

An **absolute path** begins with `/` and identifies a location starting from the filesystem root. A **relative path** is interpreted from the current working directory.

```
$ cd ~/unix_lab
$ ls documents
$ ls ./backups
$ cd documents
$ ls ../backups
$ ls ~/unix_lab/backups
```

These last two `ls` commands refer to the same directory. By contrast, `ls backups` from inside `documents` would look for a different directory, `documents/backups`, which has not been created.

Command and filename spelling is case-sensitive on typical Unix/Linux filesystems: `Data.txt` and `data.txt` are different names.

Frequently used directory operations

```
$ cd ~/unix_lab
$ mkdir -p projects/week1/src
$ cd documents
$ cd -
$ ls -ld projects
```

`mkdir -p` creates any missing parent directories in the requested path. `cd -` returns to the previous working directory and displays its path. With `ls -ld`, information about the directory itself is shown rather than the entries inside it.

Practice

1. In `~/unix_lab`, create a directory named `research`. Inside it, create two directories named `drafts` and `final`. Verify the structure using `ls`.
2. From `~/unix_lab/research/drafts`, list `~/unix_lab/backups` using (a) a relative path and (b) a path starting with `~`.
3. Explain the difference between `cd ..`, `cd .`, and `cd`.

UNIT 2 | File operations and text searches

Creating the working text file

The file-processing commands in this unit use a small text file created directly in the terminal. Run the command below and enter the twelve lines exactly as shown. After the final line, press **Enter** and then **Ctrl+D** to indicate the end of input. Do not type the text `Ctrl+D` into the file.

```
$ cd ~/unix_lab
$ cat > schedule.txt
Monday 09:00 lecture
Tuesday 10:00 lab
Wednesday 11:00 review
Thursday 09:00 lecture
Friday 13:00 project
Monday 14:00 office hours
Tuesday 15:00 lab
Wednesday 16:00 discussion
Thursday 10:00 project
Friday 11:00 review
Saturday 12:00 study
Sunday 12:00 study
[press Ctrl+D]
```

`cat` reads input and reproduces it as output. With no input filename it reads the keyboard; the operator `>` sends its output into `schedule.txt`. This creates the file or **replaces its previous contents**. Output redirection is examined systematically in Unit 3.

Copying, moving, and deleting

```
$ cp schedule.txt schedule-copy.txt
$ mv schedule-copy.txt backups/schedule.bak
$ cp schedule.txt documents/schedule.txt
$ mv documents/schedule.txt documents/weekly.txt
$ ls
$ ls backups
$ ls documents
```

`cp source destination` copies data, retaining the source. `mv source destination` relocates or renames the entry; the original path normally disappears. If the destination is an existing directory, the source keeps its filename in that directory. In the first `mv` command, a new name is explicitly supplied.

The dot may be used as a destination to mean the current directory: `cp ~/unix_lab/schedule.txt .`. Both `cp` and `mv` may overwrite existing files; verify the intended paths before using them.

```
$ cp schedule.txt temporary.txt
$ rm temporary.txt
$ mkdir disposable
$ rmdir disposable
```

`rm` removes a file. `rmdir` removes an **empty** directory and fails for a nonempty one. Removal is not ordinarily reversible through a graphical recycle bin. Avoid using `rm` with unverified paths or wildcard patterns.

Editing a file

Text files can be created with `cat > filename`, as above, or edited using a terminal editor. For example, `nano notes.txt` opens the file in `nano` when that editor is installed. Enter the text, press **Ctrl+O** followed by **Enter** to save, and press **Ctrl+X** to exit. This differs from **Ctrl+D**, which signals end-of-input when entering text through `cat`.

```
$ nano notes.txt
$ cat notes.txt
```

To recall earlier commands, press the **Up Arrow** at the shell prompt. In Bash, `history` displays saved command history; the number and availability of entries depend on the shell session and its configuration.

```
$ history
```

Displaying file contents

```
$ cat schedule.txt
$ less schedule.txt
$ head schedule.txt
$ head -n 4 schedule.txt
$ tail schedule.txt
$ tail -n 3 schedule.txt
$ clear
```

`cat` prints all lines; `less` provides a paged viewer for longer files. Within `less`, use **Space** to advance, **q** to quit, `/project` then **Enter** to search, and **n** for the next match.

`head` prints the first ten lines by default; `tail` prints the last ten. With `-n N`, either command prints N lines. `clear` clears the terminal display without changing any files.

Finding text with grep and counting with wc

```
$ grep 'Monday' schedule.txt
$ grep -i 'LAB' schedule.txt
$ grep -n 'project' schedule.txt
$ grep -v 'lecture' schedule.txt
$ grep -c 'lab' schedule.txt
$ grep -in 'MONDAY' schedule.txt
$ wc -l schedule.txt
$ wc -w schedule.txt
$ wc -c schedule.txt
```

`grep pattern file` prints lines containing a matching text pattern. Matching is case-sensitive unless `-i` is specified. Quotes keep multiword patterns together (for example, `grep 'office hours' schedule.txt`).

Option	Effect
`-i`	Ignore letter case.
`-n`	Prefix each matching line with its line number.
`-v`	Select lines that do *not* match.
`-c`	Show the number of matching lines, not the lines themselves.

Options may be combined, as in `grep -ivc 'lab' schedule.txt`, which counts lines that do not contain `lab` in any letter case. `wc` counts lines (`-l`), words (`-w`), or bytes (`-c`). The file `schedule.txt` contains twelve lines.

Searching subdirectories

The `-r` flag makes `grep` search files under a directory recursively. The following command searches files under the current practice directory for the word `lecture` and includes filenames and line numbers in results. Only text files should be used as search targets in these exercises.

```
$ cd ~/unix_lab
$ grep -rn 'lecture' .
```

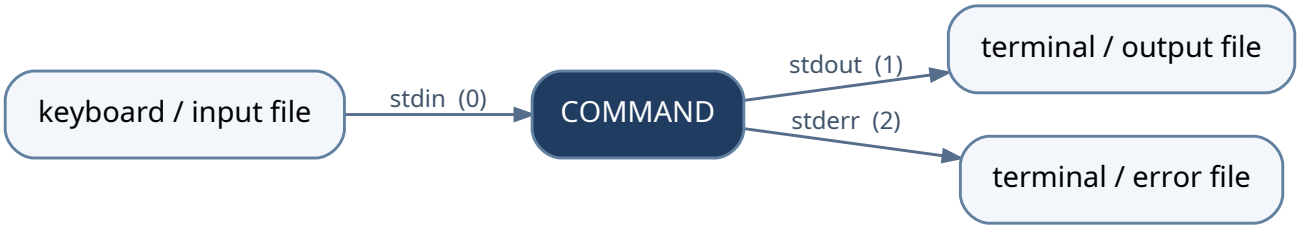
Practice

4. Make a copy of `schedule.txt` in `backups` named `schedule-copy.txt`; then rename that copy to `previous-schedule.txt`.
5. Display the last five lines of `schedule.txt`. Find each line containing `review`, including its line number. Count the lines containing `Monday`.
6. Create an empty directory named `old_notes`; remove it. Explain why `rmdir backups` is expected to fail in this workspace.

UNIT 3 | Input, output, redirection, and pipes

Standard streams

A command normally receives input through **standard input** (stdin, descriptor 0), sends ordinary results to **standard output** (stdout, descriptor 1), and sends diagnostic messages to **standard error** (stderr, descriptor 2). By default, stdin is associated with the terminal keyboard, while stdout and stderr appear on the terminal display.



Standard input, standard output, and standard error.

For example, enter `cat` without arguments. Each line typed on the keyboard is sent back to stdout. Pressing **Ctrl+D** after a completed line signals the end of input; it is not a request to terminate a process by force.

```
$ cat
first line
second line
[press Ctrl+D]
```

Output and input redirection

The operator **>** directs stdout into a file, creating it if needed and **truncating it if it exists**. The operator **>>** appends stdout to a file. The operator **<** makes the contents of a file available on stdin.

```
$ cat > cities1.txt
Mobile
Chicago
Austin
Boston
[press Ctrl+D]
$ cat > cities2.txt
Denver
Mobile
Dallas
[press Ctrl+D]
$ cat >> cities1.txt
Atlanta
Seattle
[press Ctrl+D]
$ cat cities1.txt cities2.txt > all_cities.txt
$ cat all_cities.txt
```

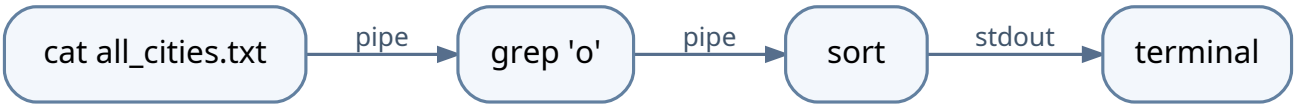
The two inputs are combined in the order given: `all_cities.txt` contains the six entries from `cities1.txt`, followed by the three from `cities2.txt`. Repeating the final command with **>** would replace the existing output, not append another copy.

```
$ sort < all_cities.txt
$ sort < all_cities.txt > sorted_cities.txt
$ cat sorted_cities.txt
```

`sort` sorts text lines lexicographically by default. The first command obtains input from the file and displays the ordered lines; the second saves them. For numerical input, `sort -n` compares lines as numbers rather than as text. `sort all_cities.txt` is another way to supply a file to `sort`.

Pipes

A pipe, written `|`, connects stdout of the command on its left to stdin of the command on its right. This allows commands to process data without a temporary file.



A pipeline passes one command's output to the next command.

```
$ cat all_cities.txt | grep 'o' | sort
$ grep 'Mobile' all_cities.txt | wc -l
$ who | sort
$ who | wc -l
```

The first pipeline finds lines containing lowercase `o` and sorts those selected lines. The second counts lines containing `Mobile`. The command `who` reports currently logged-in sessions; `who | wc -l` counts the output records, not necessarily distinct people. On some systems the result may be zero.

Pipeline stages run as a connected sequence of processes; the output of one stage becomes the next stage's input. Ordinary pipe connections carry stdout, **not stderr**.

Redirecting error messages

```
$ ls file_that_does_not_exist > listing.txt
$ ls file_that_does_not_exist 2> errors.txt
$ cat errors.txt
```

In the first command, `listing.txt` is created but the error message still appears on the terminal because `>` redirects stdout only. In the second command, `2>` redirects stderr to `errors.txt`. This distinction is important when troubleshooting commands or saving their results.

Exit status

Every completed command has an exit status. A value of `0` conventionally indicates success; a nonzero value indicates that some error or exceptional condition occurred. The shell variable `$?` holds the exit status of the most recently completed command. It must be checked immediately, before another command changes it.

```
$ ls schedule.txt
$ echo $?
0
$ ls nonexistent_file
ls: cannot access 'nonexistent_file': No such file or directory
$ echo $?
2
```

The nonzero value in the example is typical of GNU `ls`; other commands may use different nonzero codes. Later, `make` uses command failures to decide whether a build succeeded.

Practice

7. Create a text file `numbers.txt` with the lines `12`, `3`, `21`, and `7`. Compare `sort numbers.txt` with `sort -n numbers.txt`.
8. Using one pipeline, display the names in `all_cities.txt` containing `a`, sorted alphabetically. In another pipeline, count the lines containing `Mobile`.
9. Explain why the operators `>`, `>>`, `<`, `|`, and `2>` produce different effects.

UNIT 4 | Filename patterns and documentation

Shell filename patterns

The shell can expand patterns into matching filenames before it starts a command. These patterns are called **filename globs**. They match names in the filesystem; they do not search the text *inside* files. For a controlled example, create the following empty files:

```
$ cd ~/unix_lab
$ touch lab1.c lab2.c lab10.c report1.txt report2.txt reportA.csv
```

`touch` creates an empty file if the named file does not already exist. If a file already exists, it normally updates its timestamps without replacing its contents.

Pattern	Meaning	Example
*	Zero or more characters.	`ls *.c` lists names ending in `.c`.
?	Exactly one character.	`ls lab?.c` matches `lab1.c` and `lab2.c`, not `lab10.c`.

```
$ ls report*.txt
$ ls lab?.c
$ ls lab*.c
$ ls *.txt
```

The first pattern matches `report1.txt` and `report2.txt`, but not `reportA.csv`. On typical shells, `*` and `?` do not match initial dots unless the pattern explicitly begins with a dot. If no filename matches, shell behavior can vary, so an unmatched pattern should not be treated as proof that a command succeeded.

Do not confuse `ls *.txt` with `grep 'txt' file`: the former matches filenames; the latter matches lines of file content.

Filename conventions

File and directory names should be descriptive and consistent. Letters, digits, underscores, hyphens, and periods are convenient choices. A common suffix indicates file type, for example `.txt` for plain text or `.c` for C source code.

Spaces are legal in filenames but require quoting or escaping in commands. Shell-special characters such as `*`, `?`, and `&` should not be used casually in names, because the shell may interpret them as operators.

```
$ touch 'draft report.txt'
$ mv 'draft report.txt' draft_report.txt
$ ls draft_report.txt
```

The quotes in the first two commands ensure that the space belongs to one filename rather than separating two command arguments. The final name avoids this quoting requirement.

Command documentation

```
$ man wc
$ whatis wc
$ apropos copy
$ man grep
```

`man command` opens a manual page containing the command's description, syntax, and options; press **q** to quit. `whatis command` requests a brief description. `apropos keyword` searches short manual descriptions for a topic. The availability of `whatis` and `apropos` depends on the manual database installed on the machine.

Manual notation often places optional arguments in brackets. These brackets are explanatory notation, not characters to type unless the manual explicitly says otherwise. For example, a command description might show `head [OPTION]... [FILE]...`; an actual invocation is `head -n 5 schedule.txt`.

Practice

10. Identify exactly which of the three C source files are selected by `lab?.c`, `lab*.c`, and `*.c`.

11. Use a glob to list the text files in `~/unix_lab`. Explain why some files in the `documents` and `backups` subdirectories are not included.

12. Find the manual for `sort` and identify the option that performs numerical sorting. State the difference between `man` and `whatis`.

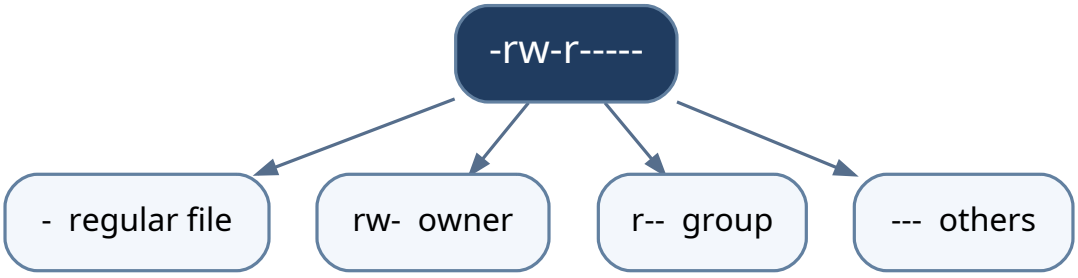
UNIT 5 | Permissions, processes, and jobs

Reading access permissions

Each file and directory has a user owner, a group owner, and a set of permission bits. The long listing format displays this information. The following is an *example*; ownership, size, and date on the current computer will differ.

```
$ cd ~/unix_lab
$ ls -l schedule.txt
-rw-r----- 1 student class 253 Oct 13 10:40 schedule.txt
$ ls -ld backups
```

The first character describes the entry type: `-` is a regular file and `d` is a directory. The next nine characters form three groups of permissions: owner, group, and others.



File type and permission groups.

`ls -l backups` lists the *contents* of `backups`; `ls -ld backups` shows the permissions of the *directory itself*.

Permission	Regular file	Directory
`r` (read)	Read the contents.	List entry names (when traversal is also allowed).
`w` (write)	Modify the contents.	Create, remove, or rename entries, normally requiring `x` too.
`x` (execute)	Execute as a program where applicable.	Traverse the directory or access a known entry name.

To reach a file through a pathname, the user needs directory traversal permission (`x`) on every directory in the path. To read file contents, the file itself also needs an applicable `r` permission. **Deleting a file depends primarily on write and traversal rights to its parent directory, not on the file's own write bit.**

Changing permissions with chmod

`chmod` changes permissions on a file or directory, subject to ownership and system rules. In symbolic form, `u` means owner, `g` group, `o` others, and `a` all. `+` adds a permission, `-` removes one, and `=` assigns exactly the indicated permissions.

```
$ chmod go-rwx schedule.txt
$ ls -l schedule.txt
$ chmod u=rw,g=r,o= schedule.txt
$ ls -l schedule.txt
```

The second permission command gives the owner read and write rights, the group read rights, and others no rights. Symbolic operations affect the specified categories; categories not named are left unchanged.

Permissions may also be expressed numerically. Each group uses `r = 4`, `w = 2`, and `x = 1`; add the values to obtain a digit.

Mode	Owner	Group	Others	Typical interpretation
600	rw-	---	---	Private, owner-editable file.
644	rw-	r--	r--	Owner-editable, others-readable file.
700	rwx	---	---	Private directory with owner access.
755	rwx	r-x	r-x	Traversable directory visible to others.

```
$ chmod 600 schedule.txt
$ chmod 700 backups
$ ls -l schedule.txt
$ ls -ld backups
```

Only change permissions on files and directories you own or are authorized to manage. Permissions on a directory may prevent access to files even if those files themselves are readable.

User and group information

The owner and the group shown by `ls -l` identify two permission categories. `id` reports the user identity and group identifiers. `groups` lists group memberships. On systems that support it, `chgrp GROUP file` changes a file's group, provided the user has the required authorization and belongs to the destination group. The group name must be replaced with a group that is actually available on the system.

```
$ id
$ groups
$ ls -l schedule.txt
```

`chmod` changes permission bits, not ownership. Changing the group is not necessary for any of the exercises in these notes.

Processes and job control

A **process** is a running instance of a program and has a process identifier (**PID**). The shell also assigns **job numbers** to commands that it manages. A job may be in the foreground, background, or stopped state. The `ps` command shows processes; `jobs` shows jobs tracked by the current shell.

```
$ ps
$ sleep 5
$ sleep 120 &
$ jobs
```

`sleep N` waits N seconds and then exits. In the foreground, the shell generally does not issue another prompt until the command finishes. Appending `&` starts a background job and returns the prompt. Its reported job number might be `[1]`, while its PID is a different number. Job numbers vary between sessions.

```
$ sleep 300
[press Ctrl+Z]
$ jobs
[2]+ Stopped  sleep 300
$ bg %2
$ jobs
$ fg %2
[press Ctrl+C]
```

Ctrl+Z requests that the foreground job be stopped. `bg %2` resumes job 2 in the background; `fg %2` brings it to the foreground. **Ctrl+C** sends an interrupt to the foreground job, usually terminating it. If the displayed job number is not 2, substitute the number shown by `jobs`. Do not place an interactive program in the background if it needs continued keyboard input.

Terminating processes

```
$ sleep 300 &
$ jobs
[3]+ Running  sleep 300 &
$ ps
$ kill %3
$ jobs
```

`kill %1` sends a termination signal to job 1 in the shell's job table. To address a process by PID instead, use `kill 12345`, substituting the actual PID obtained from `ps` or the job-start message. Neither a job number nor a PID should be guessed.

Ordinary `kill` sends **SIGTERM**, which permits a process to handle a termination request. `kill -9 PID` sends **SIGKILL**, which cannot be handled and should be reserved for processes that do not respond to ordinary termination. A user normally cannot terminate another user's processes without appropriate authorization.

The control keys have different meanings: **Ctrl+D** ends terminal input, **Ctrl+Z** stops a foreground job, and **Ctrl+C** interrupts a foreground job.

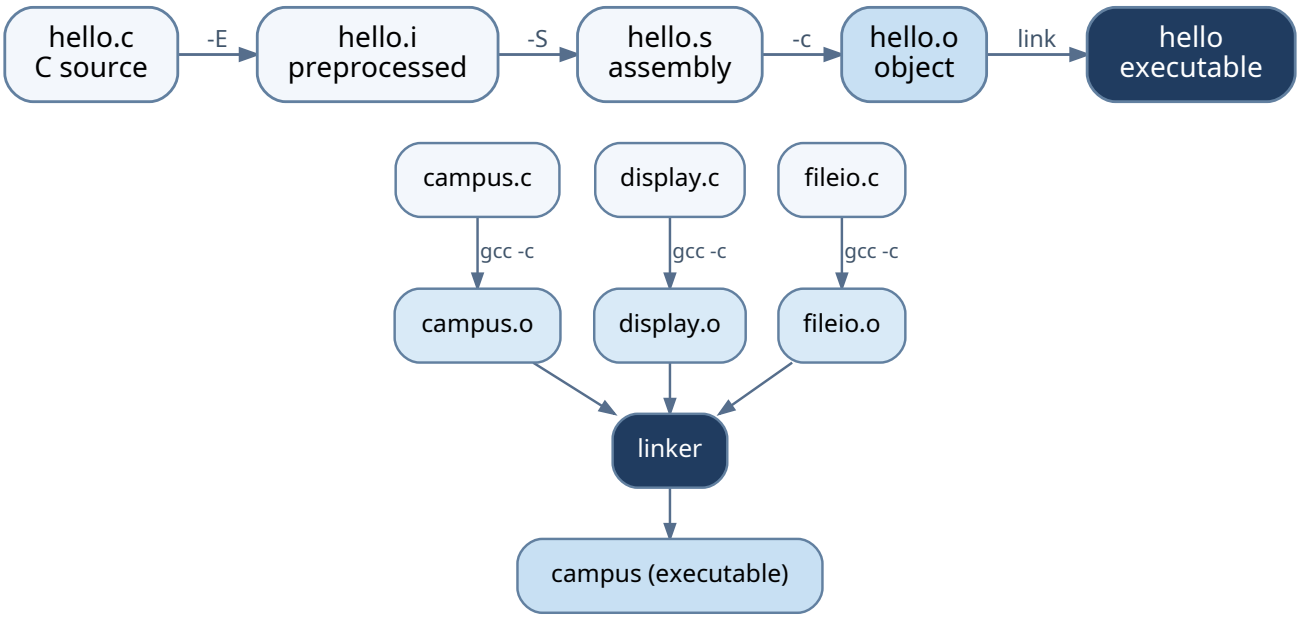
Practice

13. Set `schedule.txt` to mode `640`. Write out the owner, group, and other permission bits and verify using `ls -l`.
14. Create `private` inside `~/unix_lab` and set its mode to `700`. Verify its mode using `ls -ld private`. Explain why directory `x` is important.
15. Start `sleep 300` in the background, inspect it with `jobs` and `ps`, and terminate it with `kill`. Repeat using `Ctrl+Z`, `bg`, and `fg`.

UNIT 6 | Compiling C programs and using make

The compilation pipeline before Makefiles

Before using `make`, it is important to understand how a C program becomes an executable. The build process has four conceptual stages: preprocessing, compilation, assembly, and linking. The first three stages may be applied to each source file separately; linking combines the resulting object files into one executable program.



Separate object files are linked to form one executable.

The stages have different roles. **Preprocessing** handles directives such as `#include` and `#define`. **Compilation** translates preprocessed C code into assembly language. **Assembly** converts assembly language into machine-code object files. **Linking** combines object files and libraries, resolving cross-file function calls and producing the executable.

For a first demonstration, create a small program that contains its own `main` function and does not depend on any other source file:

```
$ cd ~/unix_lab
$ mkdir -p build_demo
$ cd build_demo
$ cat > hello.c
#include <stdio.h>

int main(void) {
    printf("Hello from C\n");
    return 0;
}
[press Ctrl+D]
```

The GNU C compiler, `gcc`, can stop after a chosen stage. The following commands show the important outputs and the flags commonly used in an introductory course:

Command	Meaning
`gcc -E hello.c -o hello.i`	Run only the preprocessor. The output `hello.i` is expanded C source after processing directives.
`gcc -S hello.c -o hello.s`	Run preprocessing and compilation, then stop before assembly. The output `hello.s` is assembly code.
`gcc -c hello.c -o hello.o`	Run preprocessing, compilation, and assembly, then stop before linking. The output `hello.o` is an object file.
`gcc hello.c -o hello`	For a single source file, run all required stages and produce the executable `hello`.
`gcc campus.o display.o fileio.o -o campus`	Link existing object files to produce the executable. No C source is compiled in this command.

```
$ gcc -E hello.c -o hello.i
$ gcc -S hello.c -o hello.s
$ gcc -c hello.c -o hello.o
$ gcc hello.o -o hello
$ ./hello
Hello from C
$ ls hello*
```

The `.i`, `.s`, and `.o` files are generated by the compiler. The final command that links `hello.o` produces the runnable executable.

The option `-o name` gives the name of the output file. Without `-c`, `gcc` normally attempts to continue to the linking stage. Thus object files such as `campus.o` come from earlier compilation steps; they are not written by hand. This point is essential for understanding what a Makefile must specify.

In the examples that follow, the simplest command, `gcc campus.c display.c fileio.c -o campus`, compiles the source files and links them in a single invocation. Explicit object files become useful when only part of a multi-file program changes, because unchanged files need not be recompiled.

A complete program with three source files

Create a separate build directory. The following files are written using `cat`; type the lines between the command prompt and the end-of-input instruction, then press **Ctrl+D**. The declarations of `show_banner` and `show_course` tell the compiler about functions defined in different source files.

```
$ cd ~/unix_lab
$ mkdir -p build_demo
$ cd build_demo
$ cat > campus.c
#include <stdio.h>

void show_banner(void);
void show_course(void);

int main(void) {
    show_banner();
    show_course();
    return 0;
}
[press Ctrl+D]
```

```
$ cat > display.c
#include <stdio.h>

void show_banner(void) {
    printf("Campus tools\n");
}
[press Ctrl+D]
```

```
$ cat > fileio.c
#include <stdio.h>

void show_course(void) {
    printf("Course: CSC 210\n");
}
[press Ctrl+D]
```

The two files `display.c` and `fileio.c` contain function definitions. `campus.c` contains `main` and the function declarations. The program prints a fixed message to make the effects of compilation and linking easy to observe.

```
$ gcc campus.c display.c fileio.c -o campus
$ ./campus
Campus tools
Course: CSC 210
$ ls
```

`-o campus` sets the output filename. The executable is started with `./campus` because the current directory is not ordinarily searched automatically for commands. No `.o` files need be saved by the user in this one-command method. They become explicit in a later example.

The first Makefile: one command

A `Makefile` is a text file containing rules. By default, GNU `make` reads `GNUmakefile`, `makefile`, or `Makefile` in the current directory. The usual convention in these notes is the capitalized name `Makefile`.

Create a new file named `Makefile` containing exactly the following two lines. The second line must start with a real **Tab** character, not spaces.

```
all:
<TAB>gcc campus.c display.c fileio.c -o campus
```

Here and in the remaining Makefile examples, `<TAB>` is an instruction to press the Tab key **once**; those six printed characters are not part of the file. The line `all:` names a target. The indented command is its **recipe**. Use `nano Makefile` to create and edit this file and then save it with **Ctrl+O**, **Enter**, **Ctrl+X**.

```
$ make
$ ./campus
$ make all
```

Both `make` and `make all` run the rule named `all` because it is the first target. At this stage `all` has no prerequisites, and no file named `all` is created, so the compiler command runs every time. The rule does not yet track which files changed.

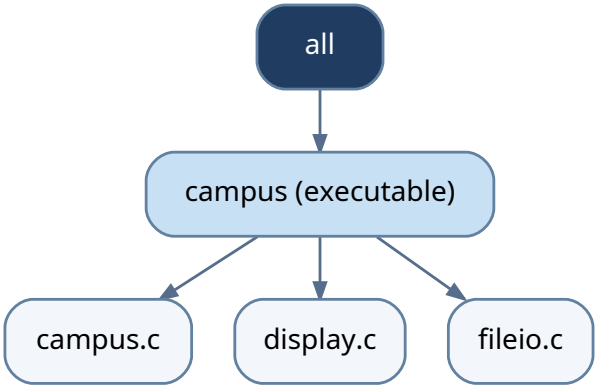
Dependencies and unnecessary rebuilding

A rule has the general form `target: prerequisites` followed by one or more tab-indented recipe lines. When a target file is missing or older than one of its prerequisites, `make` executes the recipe to update it. Replace the previous Makefile with:

```
all: campus

campus: campus.c display.c fileio.c
<TAB>gcc campus.c display.c fileio.c -o campus
```

The first rule asks `make` to build `campus`. The second rule says that the executable depends on three C files. If none of the source files has changed since the executable was generated, `make` does not compile again. If any one changes, the compiler runs again for all three source files. The dependency graph is:



Makefile dependencies when all sources compile together.

```
$ make
make: Nothing to be done for 'all'.
$ nano fileio.c
$ make
$ ./campus
```

When editing `fileio.c`, change `CSC 210` in the `printf` string to `CSC 211`, save, and run `make` again. The exact status wording is GNU Make output; it depends on file timestamps and the current build state.

Compiling object files separately

A different compilation approach keeps a separate object file for each C file. If only `fileio.c` changes, only `fileio.o` needs recompilation; the linker then constructs a new executable. First execute the commands manually:

```
$ gcc -c campus.c
$ gcc -c display.c
$ gcc -c fileio.c
$ ls *.o
campus.o  display.o  fileio.o
$ gcc campus.o display.o fileio.o -o campus
$ ./campus
```

The `-c` option stops before the linking stage. It saves object code in a corresponding `.o` file. The last `gcc` command links the three object files; it does not compile their `.c` sources.

The following Makefile is **incomplete**. It illustrates a frequent mistake:

```
all:
<TAB>gcc campus.o display.o fileio.o -o campus
```

```
$ rm -f campus.o display.o fileio.o
$ make all
```

The second command fails during linking because the three required object files do not exist. The diagnostic includes messages such as `cannot find campus.o`. This Makefile specifies what to run but not how to produce the missing inputs. Earlier commands that compiled the objects manually are not remembered by `make`.

A correct Makefile with explicit rules

Replace the incomplete file with the following. It names the final executable as a target and gives each required object file its own compilation rule.

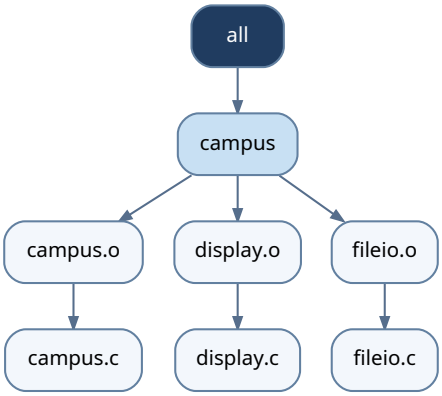
```
all: campus

campus: campus.o display.o fileio.o
<TAB>gcc campus.o display.o fileio.o -o campus

campus.o: campus.c
<TAB>gcc -c campus.c

display.o: display.c
<TAB>gcc -c display.c

fileio.o: fileio.c
<TAB>gcc -c fileio.c
```



Makefile dependencies when each source has its own object file.

Now `make all` first checks the prerequisites of `campus`. Missing object files are built from their respective source files, after which linking occurs. When only `fileio.c` changes, `make` recompiles `fileio.o` and relinks `campus`; it does not unnecessarily recompile the two unchanged source files.

```
$ rm -f campus campus.o display.o fileio.o
$ make all
$ make all
$ make -n all
```

`rm -f` removes the named build outputs without reporting an error when they do not exist. The first `make all` reconstructs them; the second normally reports that there is nothing to do. `make -n all` displays the commands that would be executed without running them (with special behavior for recursive make, not needed here).

Adding a clean target

A target can represent an action rather than a generated file. Extend the Makefile by appending:

```
.PHONY: all clean

clean:
<TAB>rm -f campus campus.o display.o fileio.o
```

`make clean` runs the cleanup command. The `clean` target is **not** part of the dependencies of `all`; cleaning must therefore be explicitly requested. `.PHONY` states that `all` and `clean` are action targets, not ordinary files. If a file happened to be named `clean`, declaring it phony prevents that file from blocking the recipe.

```
$ make clean
$ ls *.c
$ make
$ ./campus
```

Cleaning removes generated object files and the executable, **not** the source files or the Makefile. Thus `make` can rebuild the entire program after `make clean`.

Introducing a header file

The program currently declares the same shared functions in `campus.c`. A header can collect their declarations in one place and be included by the source files. This is useful when several source files must agree on the function interfaces. Create a file named `campus.h`:

```
$ cat > campus.h
void show_banner(void);
void show_course(void);
[press Ctrl+D]
```

Open each of the three `.c` files with `nano`. Add `#include "campus.h"` below `#include <stdio.h>`. In `campus.c`, remove the two separate `void ...;` declaration lines, because they now come from the header. No `#define` directive is required for this example.

The complete revised `campus.c` is:

```
#include <stdio.h>
#include "campus.h"

int main(void) {
    show_banner();
    show_course();
    return 0;
}
```

The following is the complete final Makefile. Compared with the previous version, the three object-file rules now list `campus.h` as an additional prerequisite:

```
all: campus

campus: campus.o display.o fileio.o
<TAB>gcc campus.o display.o fileio.o -o campus

campus.o: campus.c campus.h
<TAB>gcc -c campus.c

display.o: display.c campus.h
<TAB>gcc -c display.c

fileio.o: fileio.c campus.h
<TAB>gcc -c fileio.c

.PHONY: all clean

clean:
<TAB>rm -f campus campus.o display.o fileio.o
```

```
$ make clean
$ make all
$ ./campus
```

The `#include` directives tell the compiler to read declarations from the header. The Makefile prerequisites tell `make` when it must recompile an object file. These are different responsibilities: including a header does **not** automatically make an explicit Makefile rule aware of changes to that header. When `campus.h` changes, all three object files are rebuilt with these rules.

Executing make and diagnosing common errors

Symptom	Cause and correction
`missing separator`	A recipe line uses spaces instead of a leading Tab. Insert a real Tab.
`No rule to make target ...`	A prerequisite is unavailable, and no rule can build it. Check the spelling and paths.
`cannot find campus.o`	A linking command ran before object files existed. Add prerequisite rules, or compile objects first.
`Nothing to be done`	Requested targets exist and are not older than their prerequisites; or there is no work for that goal.
`undefined reference`	The linker could not find a required function definition; check sources, signatures, and linked objects.

A Makefile is a filename, not an option to `make`. From the directory containing `Makefile`, run `make` (default target) or `make all` (named target). `make -f other.mk` is needed only if a different file must be selected. For example:

```
$ cd ~/unix_lab/build_demo
$ make
$ make clean
$ make all
$ make -f Makefile all
```

The final line is valid but redundant because `Makefile` is already the default filename. If `make` says no targets are available, verify both the current directory with `pwd` and the filename with `ls -l Makefile`.

Practice

16. Explain the difference between `gcc campus.c display.c fileio.c -o campus`, `gcc -c campus.c`, and `gcc campus.o display.o fileio.o -o campus`.
17. Starting from no generated files, use `make all` to produce the executable, verify the output, and explain why running `make all` again usually does not invoke GCC.
18. Modify only `display.c`. Predict which compilation and linking commands run with the correct object-file Makefile. Verify by executing `make`.
19. Remove `fileio.o` but leave the other two object files. Predict what `make` does. Verify the prediction.
20. After introducing `campus.h`, change its modification time with `touch campus.h`. Explain why all three objects rebuild if the header is listed as a prerequisite of each one.
21. Explain what is removed by `make clean`. Identify the purpose of the Tab character, `.PHONY`, and the difference between `make` and `make -f alternate.mk`.

Command reference

The reference summarizes the commands and shell operators introduced in these notes. A filename shown as `file` or `directory` is a placeholder that must be replaced by an actual path.

Command / notation	Purpose
`pwd`; `ls`; `ls -a`	Show location; list visible entries; include dotfiles.
`mkdir directory`; `cd path`	Create a directory; change directory.
`mkdir -p path`; `cd -`	Create missing parent directories; return to previous directory.
`.`; `..`; `~`	Current directory; parent; home directory.
`cp source dest`; `mv source dest`	Copy; move or rename.
`rm file`; `rmdir directory`	Remove file; remove empty directory.
`cat`; `less`; `clear`	Show or enter text; page through text; clear screen.
`head -n N file`; `tail -n N file`	Display first or last N lines.
`grep [options] pattern file`	Search lines (`-i`, `-n`, `-v`, `-c`).

Command / notation	Purpose
`grep -rn pattern directory`	Search recursively, reporting matching line numbers.
`wc -l/-w/-c file`	Count lines, words, or bytes.
`>`; `>>`; `<`; `2>`	Redirect stdout (replace); append; supply stdin; redirect stderr.
`command1 | command2`	Connect first command stdout to second stdin.
`sort`; `sort -n`; `who`	Sort text; sort numerically; show logged-in sessions.
`*`; `?`; `touch file`	Match zero or more; match exactly one; create empty file or update timestamp.
`man`; `whatis`; `apropos`	Consult installed command documentation.
`nano file`; `history`	Edit a text file; list Bash command history.
`id`; `groups`	Display user and group information.
`ls -l`; `ls -ld directory`	Show long listing; show directory entry itself.
`chmod MODE file`	Change permission bits, symbolically or numerically.
`ps`; `jobs`	Show process information; show shell jobs.
`command &`; `bg %N`; `fg %N`	Background a command; background/foreground an existing job.
`kill PID`; `kill %N`	Request termination by process ID or job number.
`gcc ... -o program`	Compile and link C source files, specifying the output executable.
`gcc -c file.c`	Compile/assemble to an object file without linking.
`make`; `make all`	Build the default or named goal from a Makefile.
`make -f other.mk`	Read the specified build rules file.
`make clean`	Run an explicitly defined cleanup target.